# Corpus preparation

## Eight stages, two networks, and the phase that runs before an engine sees the audio

A walkthrough of `preprocess` — grounded in `preprocess-core`, `burn-mdx` and `burn-campplus`

---

**Abstract.** — The other four papers in this directory each describe **one network**. This one does not, and saying so first is the point: `preprocess` names a **phase**, not a transformation. A trainer eats a directory of clean per-utterance clips, a recording is almost never that, and getting from one to the other is eight independent stages of which only two run a model. The organising fact is sharper than "cleaner audio trains better": `audio-kit`'s slicer cuts on silence, and a continuous music bed leaves none — so a corpus recorded behind music cannot be sliced into sentences **at all** until the bed comes off. Sixty seconds of one real stream yields 5 segments as a mixture and 14, one per utterance, once separated. That is why corpus preparation is a phase standing in front of the engines rather than a flag on one of them. Two networks carry it — MDX23C for voice-against-music and CAM++ for voice-against-voice — and six model-free stages carry the rest. Every number below is either copied from the module or example that computes it, or marked as unmeasured; the one place the record contradicts itself is named rather than smoothed over.

## Contents

1 What this document is, and what it is not . . . 2
1.1 A phase, not a transformation . . . 2
1.2 No `download`, for the same reason . . . 3
1.3 The shape every stage has . . . 3
2 The measurement everything else is a knob on . . . 4
2.1 Hysteresis is the invariant . . . 4
2.2 Streaming and batch cut in the same places . . . 5
2.3 Measuring the floor instead of guessing it . . . 5
3 `analyze`: the stage that writes nothing . . . 6
3.1 It measures nothing of its own . . . 6
3.2 Two floors, because a suggestion has to be checked . . . 6
3.3 There is no loudness column, and one of the two reasons has gone stale . . . 7
4 `clip` and `trim`: one detector, two verdicts on the middle . . . 7
5 `denoise`: averaging over time, not over frequency . . . 8
6 `normalize`: one gain, held constant . . . 8
6.1 Two targets, answering different questions . . . 8
6.2 The measurement is ffmpeg's; the gain is ours . . . 8
6.3 The gate that parses . . . 9
7 `resample`, and the workspace's one stereo exception . . . 9
8 `separate`: MDX23C . . . 9
8.1 The checkpoint **is** the architecture . . . 9
8.2 Why not the older MDX-Net v2 models . . . 10
8.3 Complex spectrum in, complex spectrum out . . . 11

8.4 The fold, the transpose, and the shape that runs ... 12
8.5 The chunk is the unit of work ... 13
8.6 The seams, and the level the model is fed at ... 13
8.7 What it does not do ... 14
9 `diarize`: CAM++ ... 14
9.1 Target-speaker extraction, not clustering ... 14
9.2 The filterbank is the contract ... 14
9.3 The mean subtraction lives in the front end, not the encoder ... 15
9.4 The network ... 16
9.5 Batching is safe, and per-file batching is not ... 17
9.6 The output is window-quantised ... 17
9.7 Where 0.55 and 3 s came from ... 18
9.8 There is no level gate, and that is a measurement ... 18
10 What has actually been measured ... 18
10.1 Coverage proves a tree, never an arithmetic ... 19
10.2 Synthetic: the check that proves the port ... 19
10.3 Real: the check that measures usefulness ... 19
10.4 The finding that is not in decibels ... 21
10.5 One place the record contradicts itself ... 21
11 Decisions, and the traps behind them ... 21
11.1 `preprocess-core` depends on `cli-kit`, and that is allowed ... 21
11.2 Both model stages refuse before they fetch ... 22
11.3 This crate was the fourth to arrive with a known bug ... 22
11.4 The shared target directory ... 22
11.5 The stage boundary is a file, deliberately ... 22
12 The recipes ... 23

# 1 What this document is, and what it is not

`docs/rvc-architecture.typ`, `docs/gptsovits-architecture.typ`, `docs/whisper-architecture.typ` and `docs/seedvc-architecture.typ` each take one network apart: what every block computes, what each loss term is for, why the training loop has the shape it does. This document covers the sixth binary, which is the one that is **not** an engine, and so it has a different shape by necessity. Two of its eight stages run a network and get the same treatment the other papers give theirs; the remaining six run no model at all, and what is worth explaining about them is not arithmetic but **what they refuse to do** — why `normalize` will never compress, why `trim` will never split, why `analyze` writes nothing.

The material is drawn from the module documentation of `preprocess-core`, `preprocess-cli`, `burn-mdx`, `burn-campplus` and `audio-kit`'s slicer, and from the two harnesses that produced the measured defaults: `burn-mdx`'s `examples/separate` and `preprocess-core`'s `examples/timbre`. Where this page and a module doc disagree, the module doc is right — it sits beside the code that would change.

## 1.1 A phase, not a transformation

Every other binary in this workspace reads a pipe when given no subcommand. That rule is emphatic — streaming is the **primary** mode of a Unix filter, and `rvc serve` was promoted to the bare invocation for exactly that reason — and read literally it obliges `preprocess` to grow one too.

**It must not.** `rvc`, `stt`, `tts` and `seedvc` each name **an engine**: one transformation, so "the engine on a pipe" is a complete description of what a bare invocation does, and there is exactly one thing it could mean. `preprocess` names **a phase**, and which stage of it to run is precisely what the subcommand chooses. A bare `preprocess` would have to pick one silently, and whichever it picked would be wrong for everybody who wanted a different one. That is not a gap waiting to be filled: adding stages makes it **worse**, and more stages are the whole reason the crate exists.

So the test to apply before adding a bare invocation is not "does this binary stream" but **is there one thing a bare invocation would mean**. A future engine still gets one, because for an engine the answer is yes. And if a **stage** here ever wants to be a filter, it gets that as a property of the stage — `preprocess denoise` reading stdin when handed no files, say — while the top level stays a chooser.

## 1.2 No `download`, for the same reason

Every engine has a `download` subcommand that fetches exactly what a default bare invocation would fetch on demand. `preprocess` has none, and the absence follows from the one above rather than being a second decision. `separate` and `diarize` fetch on demand like everything else, but a `preprocess download` would have to fetch either both models or a stage's worth, and neither is what "what a default bare invocation would fetch" means when there is no default and no bare invocation. The honest form is naming the stage whose gigabytes are being spent, which is what running it does.

## 1.3 The shape every stage has

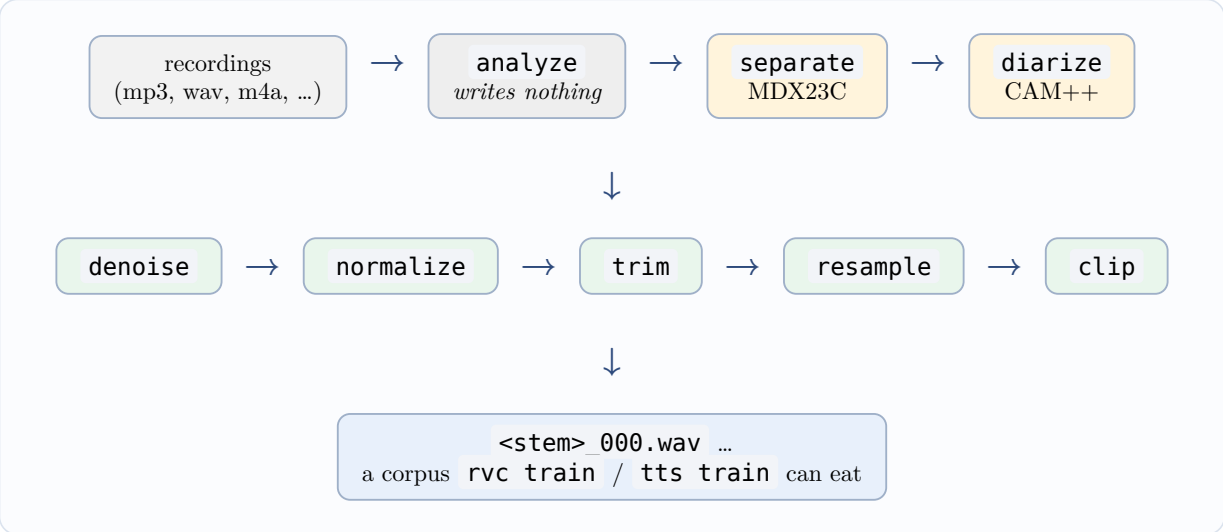


Figure 1: The phase, and where each stage sits. Only the two shaded boxes run a network; the rest are ffmpeg, arithmetic and the shared silence detector.

`plan` turns whatever paths the user named into the batch to process: directories expanded recursively to their audio files, sorted, de-duplicated, and each given an output base name that is **unique across the batch** — so `a/take.wav` and `b/take.wav` do not overwrite each other's output. Every stage then takes that list and writes into one output directory, which is what keeps the stages composable: the output of one is a legitimate input to the next.

Each stage is one module with one per-file function — `clip::file`, `denoise::file`, `separate::file`, `diarize::file`, `analyze::file`, `normalize::file`, `trim::file`, `resample::file`. Driving the batch — tolerating a file that will not decode, tallying what happened — belongs to the **caller**, because what is worth reporting differs per stage: `clip` counts clips against

input duration, `denoise` writes exactly one file per input, `diarize` counts the windows it kept against the ones it scored.

Two details of that shape are load-bearing rather than incidental:

- **No stage writes into its input directory by default.** `denoise` defaults to `denoised/`, `diarize` to `diarized/`, `trim` to `trimmed/`, `normalize` to `normalized/`, `separate` to `separated/`, `clip` to `dataset/`. A stage that overwrote its own input would make a badly chosen `--denoise-strength`, or `--threshold`, or `--silence-db`, **unrecoverable** — and every one of those knobs is one a user is expected to try twice.
- **`--sr` decides what lands on disk and nothing else.** A later stage re-decodes at whatever rate it needs, so the flag exists to avoid a resample at the end rather than to configure the analysis. `analyze` declares its own `--sr` separately for exactly this reason: it writes nothing, so what the rate decides there is the grid a measurement is taken on.

A new stage is one module in `preprocess-core`, one module under `preprocess-cli/src/commands/`, and one clap variant. That is the whole extension point, and it is the reason this is a crate rather than a second subcommand on two engines that neither of them is about — the predecessor, `preprocess-kit`, was a shared clap `Args` that `rvc` and `tts` both hosted, and it could only ever hold **one** stage, because the subcommand and the stage were the same name.

## 2 The measurement everything else is a knob on

Five of the eight stages are, underneath, one question asked differently: **where is this recording quiet?** `clip` cuts there, `trim` strips the two ends, `analyze` reports what cutting would produce, `separate` exists because a bed means the answer is "nowhere", and `diarize`'s output is judged by whether what survives can be cut afterwards. So the silence detector is the right place to start, and it lives in `audio-kit` — one definition, shared with the streaming segmenter that speech recognition cuts on, so the two cannot disagree about where a sentence ends.

### 2.1 Hysteresis is the invariant

The slicer splits a mono `f32` recording into voiced `[start, end)` ranges by removing **between-sentence dead air only**. Energy is used **solely** to locate long silent gaps — never to gate quiet-but-present sound. That distinction is the whole design, and it is there for the material this toolkit exists to preserve: in soft, breathy, close-mic content the quietest passages are **content**, not noise floor, and an energy gate deletes precisely them.

The core invariant is hysteresis on the gap: two voiced runs are one sentence unless the silence between them is **both** below the energy floor **and** longer than `min_silence`. Short internal pauses and soft tails therefore stay inside the clip, and every kept segment is edge-padded by up to `pad` seconds of the bordering quiet, so onsets and breathy tails are not clipped off.

| knob | default | decides |
|---|---|---|
| `--silence-db` | −40 dBFS | what counts as quiet at all |
| `--min-silence` | 0.3 s | how long quiet must last to be a **boundary** rather than a pause |
| `--min-clip` | 1.0 s | the shortest kept segment |
| `--max-clip` | 0 (off) | a hard cap; longer runs split at their quietest interior frame |
| `--pad` | 0.15 s | how much bordering quiet each kept segment keeps |

Figure 2: The five knobs, and what each one decides. Defaults are the slicer's own, shared by `clip`, `trim` (the first and last two only) and `analyze`.

`--min-clip` has a floor under it that a `*-kit` crate cannot see: `rvc-train` draws 0.48 s windows, and a clip shorter than one window is decoded, feature-extracted and **then** discarded with a warning. Below 0.48 s the knob therefore buys nothing and costs the analysis of every fragment it lets through. The number is deliberately **not** a constant in `audio-kit` — a shared crate that knew a trainer's window size would be depending on an engine — so it is stated in the documentation and not enforced in the code.

## 2.2 Streaming and batch cut in the same places

There are two entry points and one algorithm. `slice` takes a whole recording; `Slicer` takes it a chunk at a time and hands back each clip as soon as it provably cannot change again. They share every step past run detection, and a property test pins them to identical cuts.

What makes that free rather than a trade is that the lookahead is **bounded**: a voiced run's end can no longer move once `max(min_silence, 2·pad)` of silence has followed it, so the streaming slicer finalises there and the ranges it emits are the ones the whole recording would have given. The single divergence is speech that never pauses longer than `--max-clip`, where streaming cuts at the quietest frame in the window it has and batch balances the split across the whole run.

That equivalence is why `stt` streams its transcription today instead of buffering ten minutes before emitting a word, and it is the thing to re-run before touching either path.

## 2.3 Measuring the floor instead of guessing it

`--silence-db -40` is a dBFS number nobody can estimate about a recording they have not analysed, and on breathy close-mic material getting it wrong is exactly the failure this phase exists to prevent. So `audio_kit::noise_floor` measures it: frame the recording on the slicer's own grid (roughly 30 ms windows at a 10 ms hop, derived from the sample rate alone so the measurement never depends on the knob it is there to decide), convert each frame to dBFS, sort, and read two percentiles.

$$\text{floor}_{\text{dB}} = P_{10} \qquad \text{signal}_{\text{dB}} = P_{75}$$

The 10th percentile is between-sentence dead air, room tone and whatever the preamp contributes; the 75th is a representative speech level, not a peak. Their difference is the recording's SNR, with the floor clamped at −120 dBFS so a digitally silent file reports a large ratio rather than an unbounded one. A recording under 8 frames (≈80 ms) yields no measurement at all rather than a meaningless one.

The threshold derived from that pair is:

$$\text{silence}_{\text{dB}} = \min(0, \text{floor}_{\text{dB}} + \min(8\ \text{dB}, 0.4 \cdot (\text{signal}_{\text{dB}} - \text{floor}_{\text{dB}})))$$

Both constants are measurements rather than taste, and the second exists because the first cannot be the whole story:

- **8 dB above the floor.** Room tone is a distribution, not a level, and `floor_db` is the **middle** of the dead-air frames — a threshold sitting on it would call half the dead air voiced, and the dead air would survive into the corpus. 8 dB clears the spread without approaching speech on any recording with a normal dynamic range.
- **...but never more than 40% of the way to the speech.** A close-mic breathy take has almost no range to spend a fixed margin in. On this repository's own corpus, rebuilt into a raw recording — ten sentences with a second of the corpus's own room tone between them — the floor measures −51.4 dBFS and the speech −41.6: a **9.8 dB** gap, in which a flat 8 dB margin lands 1.8 dB **under the voice**. Taking a fraction of the measured gap instead makes the threshold structurally unable to reach the speech, and then both failure modes are bounded from the same measurement rather than by two unrelated constants.

0.4 is where a sweep put it. Every value from 0.3 to 0.6 recovers all ten sentences of that recording — against **five** for the fixed −40 dBFS — but they differ in what they keep of the soft tails: 28.5 s at 0.3, 27.9 s at 0.4, 27.0 s at 0.5 and 25.9 s at 0.6, out of a 29.7 s ceiling (all speech plus the full 0.15 s pad on each side). The fixed floor keeps 8.2 s. 0.4 keeps 94% of the ceiling while still sitting 3.9 dB clear of the tone.

**The measured floor is opt-in, and that is deliberate.** `SliceOptions::default` keeps −40 dBFS: every corpus prepared so far was cut at that number, and deriving the floor by default would silently re-cut all of them. A caller that wants the measurement asks for it, and the change gets re-baselined — the same discipline the mel front end's clip floor is held to.

It is also a **whole-signal** statistic, which is why applying it is the caller's step and not the slicer's. `Slicer` never sees the whole signal — it exists precisely so a ten-minute file need not be buffered — so measuring inside it would give the streaming path a floor that moved as audio arrived, and the two paths would cut in different places. The measurement therefore happens once, before slicing, and hands both paths the same concrete number. The honest cost is that a true stdin filter has no whole signal to measure, so this is reachable only where the audio is already in memory.

## 3 `analyze`: the stage that writes nothing

Every other stage here has at least one knob whose right value depends on the recording — `--silence-db` above all — and until this existed, nothing showed the user what the recording **was**. `analyze` is that missing half. It loads no model, takes no backend, downloads nothing and writes no audio, which is what makes it free to run over a whole corpus before deciding anything; nothing else in the binary is.

### 3.1 It measures nothing of its own

The noise floor is `audio_kit::noise_floor`, the derived threshold is `NoiseFloor::silence_db`, and the clip counts come from **running** `audio_kit::slice` rather than from a model of it. That is deliberate on both counts. A second definition of "the noise floor" is exactly what was removed when that measurement was hoisted into `audio-kit`, and a stage that **predicted** `clip`'s output instead of running its slicer would drift from it the first time either changed — while still printing numbers that looked like the truth.

### 3.2 Two floors, because a suggestion has to be checked

Each `FileReport` holds the slicer's verdict **twice**: once at the floor the user asked for, and once at the floor measured from the recording itself. Reporting only the second would make the suggestion

an assertion. Running the slicer at it is what turns it into a measurement — and it is the only way to notice the case that matters most.

**A recording with no dead air at any floor** is that case. Speech over a continuous music bed is exactly that shape: the floor sits under the music, the slicer finds nothing quiet, and no `--silence-db` can rescue it, because the quiet is not there to find. `FileReport::continuous` is that condition, and a suggested floor is **withheld** when it holds everywhere, because a floor that cannot work is worse than no floor at all. What to do about it is `separate`, which this crate also has — and pointing at it is the whole reason this stage can report a failure `clip` cannot report from inside itself.

Beyond the two slice outcomes, a file report carries duration, the noise-floor pair, peak in dBFS, a count of clipped samples, a duration histogram of the clips that would be produced, and the silence ratio — the fraction of the recording that slicing would discard. The corpus summary reduces each of those across files: median/min/max for the three level statistics, a total, the clipped-file list, and a single suggested `--silence-db` when one exists.

### 3.3 There is no loudness column, and one of the two reasons has gone stale

The reason on record is twofold. ffmpeg's `loudnorm` computes exactly what a corpus wants to know, but it reports it through `av_log` rather than through the samples, so reading it from `audio_kit::AudioFilter` would mean installing a **global** ffmpeg log callback and parsing stderr. Its `ebur128` sibling attaches the same numbers as frame metadata — **which**, the `analyze` module doc says, **`AudioFilter` discards and does not expose**. Either route being a change to a shared crate for one column is what settled it, since hand-rolling BS.1770 here would be a third definition of loudness in a toolkit that already has `NoiseFloor`.

**The second half of that is no longer true.** `AudioFilter::metadata` exists and is documented for exactly this purpose, and `normalize` reads `lavfi.r128.I` through it (Section 6) — so an integrated loudness is a function call away from this stage rather than a shared-crate change. Which of the two statements is current is not in doubt; what **is** unresolved is whether the column should now exist, and nothing in the record answers that. It is recorded here rather than reconciled, for the reason Section 10.5 gives.

What stands unchanged is that peak, floor and SNR answer "is this corpus usable" without it.

## 4 `clip` and `trim`: one detector, two verdicts on the middle

These two stages share every line of silence detection and differ in exactly one respect — what they do with the gaps **in the middle** of a recording.

`clip` cuts there. Each input becomes `<base>_<NNN>.wav` in the output directory, one file per sentence, and `--normalize` peak-normalises each written clip to 0.95 of full scale on the way out. This is the stage the trainers are documented to want first: `rvc-train`'s `sample_batch` draws random 0.48 s windows uniformly across each corpus file, so raw recordings full of between-sentence dead air collapse the generator to silence.

`trim` keeps them. Everything between the first voiced sample and the last is kept exactly as it was, pauses included — one file in, one file out, **never** a split. It carries only the two knobs that mean something without cutting (`--silence-db` and `--pad`); `--min-silence`, `--min-clip` and `--max-clip` all describe interior gaps, and this stage has no interior. It also carries the opt-in derived floor, for the recording whose room tone sits above the fixed one — where a fixed pass finds no silence to strip at all.

That `trim` reuses `audio_kit::slice` rather than looking for quiet itself is the point of it being written this way. A second silence detector would be a second set of answers to "where does this

sentence end", and the hysteresis that keeps a soft breathy tail inside a clip is the whole reason the first one is written the way it is.

## 5 `denoise`: averaging over time, not over frequency

One file in, one file out — the same `audio_kit::Denoiser` the voice-conversion filter runs on its **output**, pointed at a corpus instead. Which is the reason it is worth having as a stage: hiss that survives into the training clips is hiss the model learns to reproduce, and removing it once beforehand is cheaper and better than removing it from every conversion afterwards.

The filter is ffmpeg's `anlmdn` — non-local means over the waveform. It averages each short patch of audio with self-similar patches found **nearby in time**, so steady hiss averages away while ever-changing breath texture has few close matches and survives.

That property is the entire reason for the choice, and it is not available from the obvious alternative. Breath is spectrally almost indistinguishable from hiss: a spectral-subtraction de-noiser keyed on level and frequency has no feature that separates them, so tuning it to remove the hiss removes the breath with it. Non-local means keys on **repetition** instead, which is the one axis on which the two differ. The tuning flags live in `cli-kit`'s `DenoiseOpts`, shared with `rvc --denoise`, so the two cannot grow different spellings of `--denoise-strength`.

There is no `--denoise` switch here to go with them, unlike voice conversion's: here the stage **is** the subcommand, so a flag turning it off would leave a command that copies files.

## 6 `normalize`: one gain, held constant

One file in, one file out, at a gain that is **constant over the whole file**. That is the property the stage is built around rather than an implementation detail: a corpus of soft, breathy close-mic material is mostly quiet on purpose, and anything that moved the gain about while it played would flatten exactly the content this toolkit exists to preserve. Nothing here is a compressor, and that is not an omission to be filled in.

### 6.1 Two targets, answering different questions

- **Peak** — the loudest sample lands at a fraction of full scale, 0.95 by default. Exact, instant, and blind to everything but that one sample, so a single stray thump decides the gain for a whole recording. The 5% of headroom is there so a later resample's interpolation overshoot does not clip.
- **LUFS** — EBU R128 integrated loudness, which is what "as loud as each other" means to a listener: K-weighted, and **gated**, so the silence between sentences does not drag the reading down and the thump does not lift it.

Use peak when you want a known headroom, LUFS when you want two takes to sit at the same level. They are two `Option`s rather than one defaulted flag and one override, because **which target** is the question, and a default on both would make asking for neither mean something different from asking for peak.

### 6.2 The measurement is ffmpeg's; the gain is ours

The obvious chain is `loudnorm`, and it is the wrong one twice over. In single pass it is a **dynamic** normalizer — it compresses and true-peak limits, which is precisely the processing a breathy corpus must not get — and its linear mode needs measured values it will only ever print to a log, where an in-process filter graph cannot read them. It also negotiates its output to 192 kHz, which `AudioFilter` would hand back as if it were the requested rate.

`ebur128` has none of those problems: it passes the audio through untouched and injects the running measurement as frame metadata, so ffmpeg does the standard-compliant part and the gain stays one

multiply. The reading is taken **after** the flush, not before — the integrated value is a running one, and the figure that covers the whole recording is on the last frame out.

### 6.3 The gate that parses

R128′s absolute gate is −70 LUFS, and **ffmpeg reports the gate itself rather than `-inf` when nothing clears it**. Digital silence therefore measures exactly `-70.0`, which is finite, parses cleanly, and would be normalised **from** as if it were a very quiet recording: a two-second file of zeros handed a −23 LUFS target would receive +47 dB of gain and come out as amplified nothing. Both spellings of "nothing cleared the gate" are tested for, and the second is the one that looks like a reading.

A recording shorter than one 400 ms block has no integrated loudness at all — not a quiet one, **none** — and that is reported as its own answer rather than papered over with a peak fallback the user did not ask for.

## 7 `resample`, and the workspace's one stereo exception

Every other stage resamples on the way through, because every one of them decodes. This is the same operation with nothing else attached, and it exists so a corpus can be given one normalising step: mp3, m4a, flac and opus in, WAV out, at whatever `--sr` says.

Mono by default, because mono `f32` is what crosses every crate boundary in this toolkit and a training corpus has no use for a second channel.

The exception is worth stating rather than discovering. `separate` writes **44.1 kHz stereo** on purpose — MDX23C is stereo-native, and folding its stems to mono throws away one of the two cues it separates on — so `separate` piped into a mono `resample` silently discards that. It is only a loss if something downstream still wanted the pair, and nothing in this workspace does, since every engine decodes to mono anyway. The flag makes it a choice either way, and `--help` says which one is being made.

That single path is also the **only** place `audio-kit`'s mono rule is broken: `decode_path_stereo` → `write_wav_stereo` exists because of this network and nothing else takes `StereoSamples`. Keeping that true is the property, not a stage the workspace has yet to reach.

## 8 `separate`: MDX23C

The first of the two networks. Vocal/instrumental separation, so a corpus recorded over a music bed can be cleaned before anything else touches it: `[channels][samples]` of 44.1 kHz audio in, one waveform per stem out.

### 8.1 The checkpoint is the architecture

`dim_f`, `n_fft`, the channel widths and the block counts all differ across the MDX family, so the exact file is part of the port and not a deployment detail.

| | |
|---|---|
| **repo** | Hugging Face `Politrees/UVR_resources` |
| **revision** | `929e057b81aa49bc2e6490bef8671f47b2c120f6` |
| **weights** | `models/MDX23C/MDX23C-8KFFT-InstVoc_HQ.ckpt` (448,101,203 bytes) |
| **config** | `models/MDX23C/model_2_stem_full_band_8k.yaml` (709 bytes) |
| **stems** | `["Vocals", "Instrumental"]`, in that output order |
| **coverage** | **319 applied / 0 missing / 0 unused**, identical on every backend |

Figure 3: What `burn-mdx` is built against. The `.ckpt` is a bare `state_dict` at the root — no `state_dict` or `model` wrapper, unlike the RVC-lineage `.pth` files.

Every one of those 319 tensors is claimed by the pinned config, which is written as a constant rather than parsed from the YAML — `burn-rvc`'s `HubertConfig::chinese_base` for the same reason: a disagreeing checkpoint should fail as a shape mismatch on load, not be silently accommodated. Each field is confirmed by a tensor in the file rather than read off a document: `first_conv.weight` is `[128, 16, 1, 1]`, giving 128 channels and `dim_c = 16`; `bottleneck_block.blocks.0.tdf.2.weight` is `[8, 32]`, which is five halvings of 1024 and therefore five scales; `final_conv.2.weight` is `[32, 128, 1, 1]`, so two stems.

The stem **order** is the config's `training.instruments` list and is not recoverable from the weights at all — a caller that guesses gets the accompaniment where it wanted the voice, with nothing to indicate it.

That repository was chosen over the other mirrors because it also holds the MDX-Net v2 ONNX graphs, so one revision pins both runtimes' assets.

## 8.2 Why not the older MDX-Net v2 models

They are what UVR5 shipped first and are the obvious target, and they **cannot be Burn-ported with verifiable coverage**. They are published as ONNX only, and the graphs are BN-fused exports: roughly half of every file's 220 initializers carry no name at all, because `Conv → BatchNorm` was constant-folded and `Linear(bias=False)` became a `MatMul` over a transposed anonymous constant. Worse, one architecture has two naming schemes on disk — `UVR-MDX-NET-Inst_HQ_3.onnx` names its survivors `encoding_blocks.N.tdf.1.*` while `UVR-MDX-NET-Voc_FT.onnx` names the same tensors `BatchNormalization_460.bnu.*` — so no remap is right for both. The one `.pt` on the Hub is an `onnx2torch` TorchScript trace whose own metadata contradicts UVR's table. A coverage triple measured against any of that would be fiction.

**That is a reason to run them on ONNX Runtime, not a reason to drop them.** The `Stft` here is parameterised by `(n_fft, hop, dim_f)` precisely so it can be the front end for either: an MDX-Net v2 ONNX graph is the **whole** network, and the only thing it needs from a host is this transform. Two facts whoever wires that path will need and will not find written down anywhere else in the workspace:

- UVR keys its hyper-parameter table by the **MD5 of the last 10,240,000 bytes** of the `.onnx`, not of the whole file. Hashing the file gives a key that is in no table.
- In that table, `mdx_dim_t_set` is an **exponent**: `8` means 256 frames. `mdx_n_fft_scale_set` is `n_fft`, `mdx_dim_f_set` is `dim_f`, and `hop` is always 1024. Verified: `UVR-MDX-NET-Voc_FT.onnx` hashes to `77d07b2667ddf05b9e3175941b4454a0` → 7680/3072/8, and `UVR-MDX-NET-Inst_HQ_3.onnx` to `55657dd70583b0fedfba5f67df11d711` → 6144/3072/8.

## 8.3 Complex spectrum in, complex spectrum out

MDX23C does not consume a spectrogram **magnitude**. It consumes the complex spectrum with real and imaginary parts laid out as separate image channels — upstream calls this "cac", complex-as-channels — predicts a complex spectrum per stem, and inverts it. Phase therefore travels **through** the network rather than being reused from the mixture, which is why the inverse belongs beside the forward transform rather than in a caller.

This is deliberately **not** `burn_vits::Spectral`. That transform is a mel front end built for the VITS training objective: `center = false`, magnitudes only, 128 Slaney mel bands, and a floor that is a two-engine decision nobody may move quietly. Every one of those is wrong here. This one is `center = true` with **reflect** padding — torch's `stft` default — keeps re and im, and **truncates** the bin axis rather than warping it. The two would run happily on each other's input and compute something else, so they stay separate.

It is also host-side rather than a Burn graph, and that is a size argument. A DFT-basis matmul or a conv1d filterbank is how a **differentiable** STFT is written, and at `n_fft = 8192` the basis is `8192 × 4097 × 2` floats — 268 MB of constants for a transform that costs microseconds on a radix-2 FFT. This crate is inference-only, so it takes the FFT. That also follows the precedent RMVPE sets, whose mel front end is host-side `rustfft` in `rvc-core` and shared by both runtimes rather than reimplemented per backend — `analyze` and `synthesize` work on plain slices for exactly that reason, so a Burn model and an ONNX Runtime session cannot disagree about a transform neither of them computes.

The frame arithmetic falls out of `center = true`: padding by `n_fft / 2` on each side makes `samples` yield `samples / hop + 1` frames, and the inverse returns `hop · (frames − 1)` samples. That is what makes the checkpoint's `chunk_size = 261120` come out at exactly `dim_t = 256` frames for `hop = 1024` — the chunk size is a consequence, not an independent number.

## 8.4 The fold, the transpose, and the shape that runs

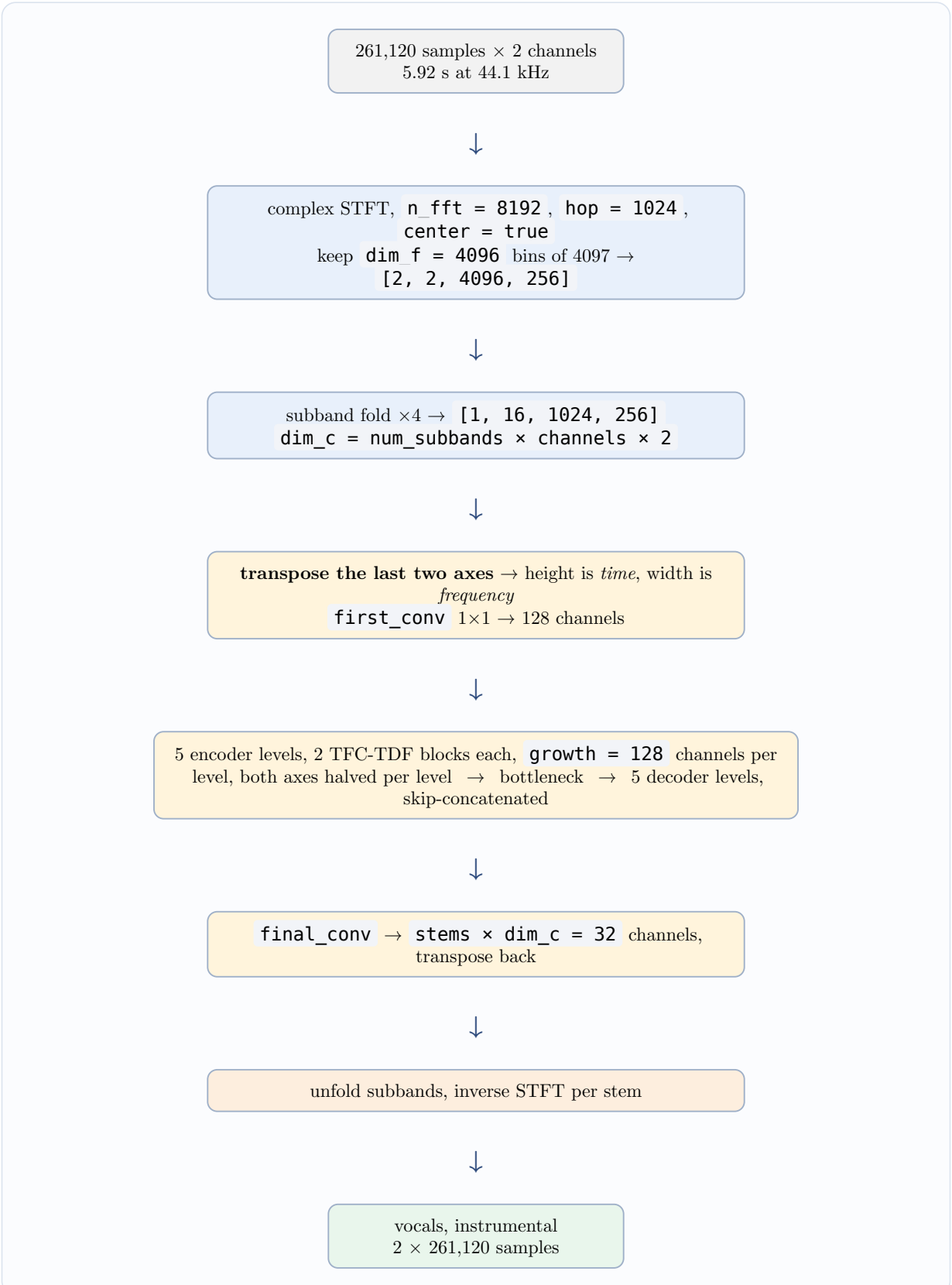


Figure 4: One chunk through the network. Every arrow but the two on the ends is a tensor of the same rank; what changes at the transpose is which axis the `Linear` in each block will act on.

Three things in that path are where a plausible port goes wrong:

**The transpose is the whole trick.** The tensor is `[batch, channels, height, width]` throughout, but which axis is which changes exactly once: the last two are swapped before the encoder and back after the decoder, so **inside every block the height is time and the width is frequency**. That is what lets the TDF's `Linear` act on the frequency axis — Burn's `Linear`, like PyTorch's, only ever touches the last one — and it is the single detail that makes a transposed port load at 100% and separate nothing.

**The norms are instance norms, and the checkpoint is the only thing that says so.** Every norm in the file carries `weight` and `bias` and **no** `running_mean` / `running_var`. Modelling them with running statistics — the shape `burn_rmvpe::unet::Norm` has, which is the obvious thing to copy — would load with two parameters missing per norm across 152 norms, and normalising by a stale running mean instead of the sample's own is silent: the output stays finite and the separation just gets worse.

**Normalisation and activation come before the convolution.** Upstream builds each TFC as `nn.Sequential(norm, act, conv)`, so the checkpoint spells the two parameterised children `.0` and `.2` — index `1` is the activation and holds nothing. A port that reads `conv → norm → act` off habit loads perfectly and computes a different function.

Within a level, one **TFC-TDF block** is a residual pair: the time-frequency convolution above, and a time-**distributed** fully-connected bottleneck across the frequency axis, `norm → GELU → Linear(f → f/4) → norm → GELU → Linear(f/4 → f)` with both linears bias-free. The shortcut is a 1×1 convolution present on **every** block, including the many where it maps a channel count onto itself — upstream builds it unconditionally, unlike RMVPE's `ConvBlockRes`, whose `hasattr` probe is why that one is an `Option`. The checkpoint confirms it: a `shortcut.weight` for all 110 blocks. Making it optional here would leave 55 parameters unloaded.

## 8.5 The chunk is the unit of work

The network's unit is one chunk of 261,120 samples — 5.92 s — and nothing in `burn-mdx` is aware of a longer recording. That is deliberate: separation runs on songs, and a caller that decodes a ten-minute stereo file into memory before starting pays several hundred megabytes before the first forward pass.

Even one chunk is not small. The first encoder level holds `[1, 128, 256, 1024]` activations — 134 MB each — and the widest decoder concatenation is 268 MB. A 6 GB GPU fits it; pure-CPU `ndarray` will run it and should not be asked to. One chunk costs around 1 TFLOP, measured at **0.83 s** on LibTorch/CUDA, which puts a real recording at roughly 3.5× realtime at 50% overlap.

## 8.6 The seams, and the level the model is fed at

`preprocess separate` owns everything `burn-mdx` refuses to know about. It decodes a stream, holds one chunk plus one hop of it, and emits finished audio as it goes, so a twenty-minute song never exists in memory as a decoded whole.

Overlap-add uses a **periodic** Hann at a half-chunk hop, which is the form that sums to one across a hop; the accumulated weight is divided out explicitly rather than relied upon, so a changed hop degrades the result gracefully instead of producing a gain ripple. A unit test pins the window pair to unity at every offset.

Level matters more than it looks. The stage runs a first pass over the stream for the **peak alone** — streamed like the second, so knowing the level costs a decode rather than a copy of the recording — and scales the whole file so that peak lands at 0.7 before the first chunk reaches the model. It has to be known up front, because a gain that changed part-way through would be a seam the crossfade cannot hide. Feeding the network audio well above the levels it was trained on degrades the separation

with no error anywhere, and the argument runs downward too, so a recording peaking at −30 dBFS is scaled **up** to 0.7 rather than left alone.

**The gain is undone before anything is written**, and that is load-bearing rather than tidiness: a stem left at the model's working level is uniformly attenuated or boosted against its own mixture, and every level a user reads off it afterwards — "how much quieter is the stem in the speech gaps" — would be wrong by exactly that gain, which reads as a separation result.

The one buffer left is on the way out, and it belongs to `write_wav_stereo` rather than to this stage: a RIFF header carries the length of what follows, so that writer collects the signal before it can write a byte. Feeding it through a channel rather than a `Vec` is deliberate — the day that writer patches its header at close instead, the stage becomes fully streaming with no change here.

### 8.7 What it does not do

It separates **voice from music**, not one voice from another. Every speaker in the recording lands in the vocals stem, including a singer on the backing track — so a streamer talking over somebody else's **singing** still gets both. Filtering by who is speaking is a different model and the next stage.

## 9 `diarize`: CAM++

Source separation cannot answer "whose voice is this", because it asks "is this a voice". Telling two speakers apart needs speaker **identity**, which is a different network: CAM++, a 192-dimensional timbre embedding, lifted into `burn-campplus` when this stage became its second reader — the same move, for the same reason, that lifted `burn-hubert` out of `burn-gptsovits`.

So the two stages compose, and the order is the whole recipe for a stream recorded over somebody else's music:

```
preprocess separate  raw/     -o vocals/   # music bed off
preprocess diarize   vocals/  -o mine/  --reference streamer.wav
preprocess clip      mine/    -o dataset/  # per-utterance clips
```

### 9.1 Target-speaker extraction, not clustering

A reference clip of the voice to keep is **required**, and `-r/--reference` being non-optional is the honest shape rather than a placeholder: blind clustering — the mode that would need no reference — is not built, and an `Option` that errored on `None` would advertise a feature that does not exist. It is also the mode worth having first. A user who wants their own speech out of a stream **has** a clean sample of it, and matching against one known voice is far more robust than discovering how many speakers a recording holds. The reference is read whole, up to 30 s, rather than windowed.

### 9.2 The filterbank is the contract

CAM++ eats a **Kaldi filterbank at 16 kHz** — 80 bins, 25 ms window, 10 ms hop, `dither = 0`, mean-normalised over time. Not the 22.05 kHz 80-band mel the vocoders speak, and not `burn_vits::Spectral`. Both are "an 80-band mel" by shape, so the wrong one loads, runs, and produces a plausible vector from a distribution the network was never trained on.

The differences that matter, each a place where substituting one transform for the other would run and compute something else:

- Per-frame DC removal, then a **preemphasis of 0.97** with a replicate pad at the frame's left edge — so the first tap keeps `1 - 0.97` of itself rather than being high-passed against a neighbour it does not have.
- The **Povey window**, `(0.5 - 0.5·cos(2πn/(N-1)))^0.85`: a **symmetric** Hann raised to 0.85, which reaches zero at both ends where a periodic Hann does not.

- The 400-sample window is zero-padded **on the right** to 512 before the transform. Right, not centred — a centred pad would rotate every frame's phase.
- `snip_edges = true` framing: no padding at either end, so the frame count is `1 + (samples − 400) / 160` and the final partial window is dropped.
- A **power** spectrum, and Kaldi's own mel scale `1127·ln(1 + f/700)` with triangles laid out evenly in that domain from `low_freq = 20` to Nyquist — not Slaney, and not area-normalised.
- The Nyquist FFT bin carries **no weight at all**: Kaldi builds the bank over `padded_window / 2` bins and torchaudio pads a zero column onto the right to reach the 257 an `rfft` returns.
- The log is floored at `f32::EPSILON`, which is what `torch.finfo(torch.float).eps` hands `torch.max`.

Waveform scale, notably, does **not** matter. Kaldi proper expects samples scaled like int16 and upstream hands it a `[-1, 1]` float waveform — but a gain of $a$ adds $2 \ln a$ to every bin of every frame, and the mean subtraction on the next line removes exactly that. The one place the scale is visible is the epsilon floor, so the port follows upstream's convention rather than Kaldi's: the floor has to sit where the released model saw it.

### 9.3 The mean subtraction lives in the front end, not the encoder

This is the trap the whole crate is arranged around. Upstream's inference writes the speaker encoder's input in two lines — the filterbank, then `feat = feat - feat.mean(dim=0)` — and **both** of them are `Fbank::forward` here.

A port written by reading the **model** is faithful and still wrong, because CAM++'s own module never normalises: the step lives in whatever code assembles its input. And the failure is silent. The embedding stays finite, stays repeatable, and quietly starts keying on the recording's channel rather than on the speaker — which looks like a mediocre conversion, or a threshold that needs tuning, and not like a bug. It was dropped **twice in one week** while Seed-VC was being built, which is why the raw filterbank is not offered at all: splitting the two would buy a caller nothing and would put the omission one forgotten line away.

Everything about the input being fixed is also unverifiable from inside. There is no numerical diff against torchaudio and there cannot be one here — this workspace runs no Python — so the semantics above are a careful reading of `torchaudio.compliance.kaldi` rather than something a test pins against the original. The unit tests check **properties**: the frame grid, DC rejection, and that a tone lands in the mel bin Kaldi's own scale puts it in. That catches a wrong frequency axis or a missing high-pass, not a window exponent of 0.8. The check that does catch those is end to end, and it is Section 9.7.

## 9.4 The network

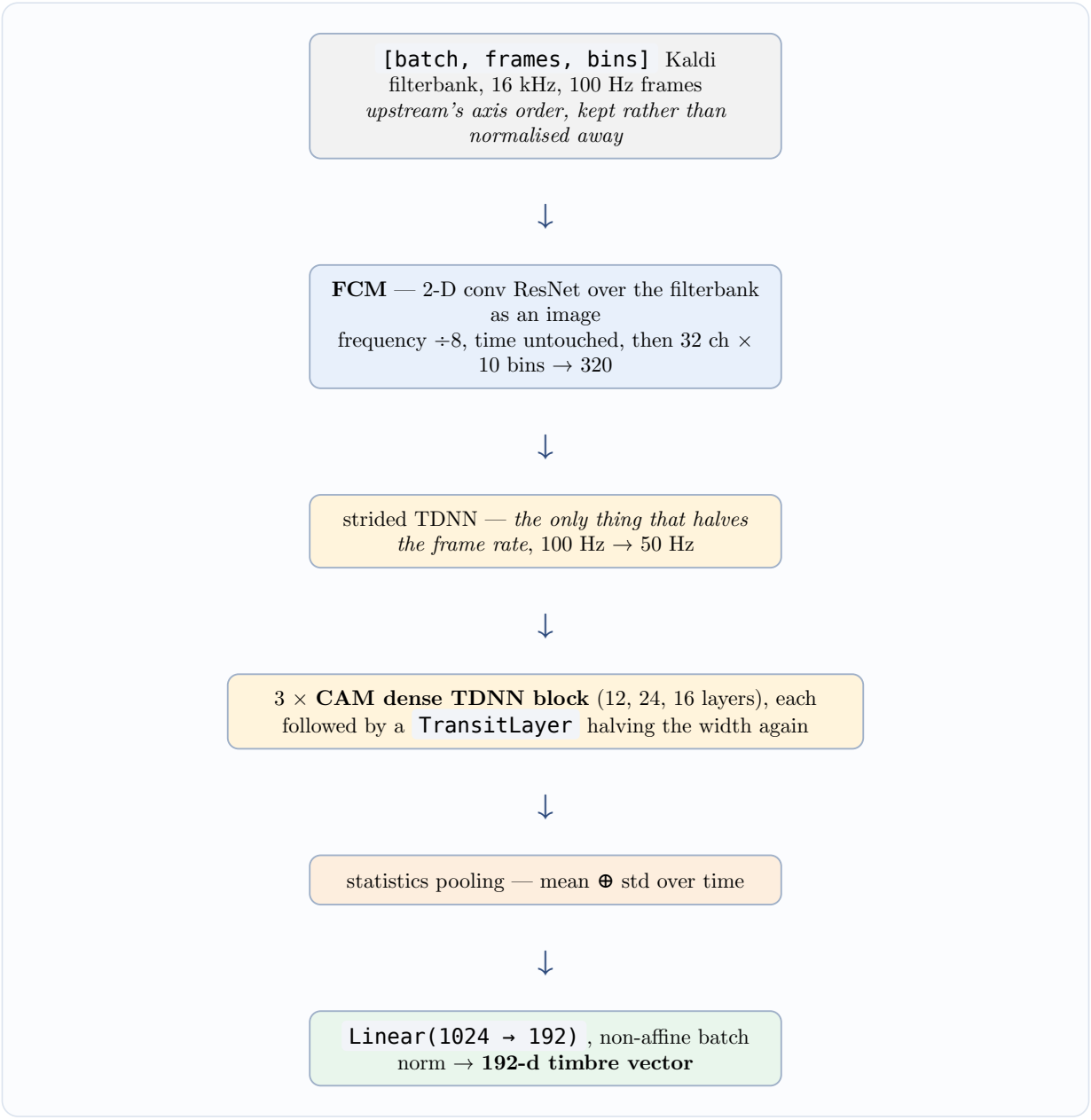


Figure 5: CAM++, `feat_dim = 80`, `embedding_size = 192`. `campplus_cn_common.bin` from `funasr/campplus` — 937 tensors, 28 MB — loads at **815 applied / 0 missing / 122 unused**, the 122 being one `num_batches_tracked` per norm. That third number is the **raw** count, not one with the expected entries already subtracted.

Two pieces deserve reading twice. **Dense** means every layer's output is concatenated onto its input, so a block's width grows by `growth_rate` per layer and the transit layer halves it again afterwards. And **context-aware masking** is the "CAM": each layer computes a local convolution and multiplies it by a gate derived from two pooled summaries — the whole utterance's mean, plus a 100-frame segment average broadcast back over time. Every frame is therefore scaled by what its neighbourhood and the utterance as a whole look like, which is how a speaker-level network suppresses content-level detail.

Statistics pooling is what makes the reference **length** irrelevant, exactly as the average pool does in Seed-VC's fossil style encoder — and it is worth being precise about what that buys, because it is not everything (see Section 9.6).

The norm always normalises by the checkpoint's stored running statistics, where PyTorch's `BatchNorm` switches on a `training` flag. Upstream calls `.eval()` before ever running this model and there is nothing here to fine-tune, so the batch-statistics branch would be dead code whose only effect could be to make an embedding depend on what else was in the batch.

Two things are worth naming as **unverified**. There is no numerical diff against upstream: `examples/load` checks that the embedding is finite, repeatable, and moves when the spectral tilt of its input moves — which catches a network that ignores its input, the failure mode that looks healthiest, but not a subtly mis-scaled one. And `eps = 1e-5` is PyTorch's `BatchNorm` default rather than anything the checkpoint records; upstream never passes an `eps`, so it follows from reading the constructor, but no tensor pins it.

#### 9.4.1 The licence, backwards from how it reads

**The weights are Apache-2.0 and this port is not.** `campplus_cn_common.bin` comes from `funasr/campplus` and the original 3D-Speaker model carries an Apache-2.0 header — but `burn-campplus` was written by reading Seed-VC's vendored copy, and Seed-VC is GPL-3.0. A port written from reading GPL source is a derivative work, so the crate inherits `GPL-3.0-only` like every other member of the workspace. A crate that must stay permissive cannot depend on it, however permissive the checkpoint it loads is. Apache-2.0 into GPL-3.0 is the compatible direction, which is what makes the combination legal — not what makes the result permissive.

### 9.5 Batching is safe, and per-file batching is not

`SpeakerEmbedder::embed` takes many clips at once, because a minute of audio is a hundred windows and a hundred separate forward passes is minutes rather than seconds. That this is **allowed** is a property of the network rather than an assumption: the norm always uses the checkpoint's running statistics, never the batch's, and the pooling that follows is per item — so an embedding cannot depend on what else was in the batch. A batch-statistics norm would have made this silently order-dependent.

The clips in one call must be the **same length**, because they are stacked into one tensor. Padding a short one out would put its silence inside the per-clip mean and then pool over it, which is wrong rather than merely different — so the caller drops a short tail instead.

**Each window is embedded from its own filterbank, never from a slice of the file's.** The front end subtracts the per-clip mean, so a mean taken over a whole recording of two speakers and a music bed would normalise every window against a distribution none of them has. Computing one filterbank per file and slicing frames out of it is roughly **fifty times cheaper** and silently wrong, which is exactly why it is named here: it is what an optimiser reaches for.

### 9.6 The output is window-quantised

The recording is cut into fixed windows, each embedded whole and scored by cosine against the reference; runs of windows that clear the threshold become the written segments. **A window that spans a speaker change belongs to whoever dominates it**, so a boundary lands on a hop rather than on the sample where one voice stopped. Two consequences worth expecting rather than discovering: a retained segment can carry a fraction of a second of the other voice at each end, and a one-word interjection shorter than a window will not be removed on its own.

The window length is the trade. Too short and the embedding is noisy — CAM++'s context-aware mask pools over 100 of its own 50 Hz frames, so below about 2 s its two context scales collapse into one. Too long and a speaker change hides inside a window. The hop is the output's resolution: segment edges land on that grid, so halving it doubles the work and halves the quantisation.

### 9.7 Where 0.55 and 3 s came from

`preprocess-core`'s `examples/timbre` is the harness, and it measures the same quantity the stage compares: one whole-reference embedding against one fixed-length **window** embedding, not clip against clip. Statistics pooling makes the embedding's **shape** length-independent, but not the cosine distribution it lands in — so a threshold calibrated on sentence-length clips would be a threshold for a different measurement.

The numbers below are what it printed for one 60 s excerpt of a stream with a song playing under it, separated first. The **vocals** stem stands in for the target speaker's material and the **instrumental** stem — the same seconds with him removed, leaving the music and the singer — for what has to be rejected. The reference is 14 s of the same speaker from a different part of the same recording, separated the same way.

| **window** | **target** p5 / p25 / median | **reject** median / p75 / p95 |
|---|---|---|
| 1.5 s | 0.215 / 0.456 / 0.562 | 0.362 / 0.449 / 0.609 |
| **3 s** | 0.374 / 0.568 / 0.628 | 0.411 / 0.464 / 0.674 |
| 5 s | 0.488 / 0.625 / 0.660 | 0.430 / 0.510 / 0.722 |

Figure 6: Cosine against the reference, by window length. At 1.5 s the two distributions do not separate at all — the target's lower quartile sits **below** the rejected material's upper one, which is the 2 s pooling window showing up as a number.

5 s separates marginally better than 3 s and quantises every boundary five seconds wide, so **3 s is the default**. At 3 s a threshold of **0.55** keeps 80% of the target's windows and rejects 86% of the music-and-singer ones.

Two honest limits, both of which bound that from above. The reference and the material share a session, so the same microphone and the same room — and CAM++'s known failure is keying on the **channel** rather than the speaker, which flatters a same-session measurement. And the rejected material is a music bed with a sung voice in it rather than a second person talking, because no clean recording of a second speaker was available; it is what this stage has to reject in the case it was built for, not a speaker-verification error rate.

The reading to take from that harness in general is the **gap**: the same-speaker 5th percentile against the different-speaker 95th. If they overlap, no threshold separates those two voices, and the honest report says so rather than picking one. That is the shape a measured default is supposed to have — a harness that can be re-run when the model changes, rather than a number somebody chose.

### 9.8 There is no level gate, and that is a measurement

A window with no voice in it embeds to something arbitrary, so the obvious guard is an RMS floor. It is not here because the threshold already covers it: over that excerpt the near-silent windows scored **0.20–0.37** against the reference, far below any threshold that keeps the target, and a −40 dBFS floor removed 3 of 58 windows that were being dropped anyway. A floor would be a second knob agreeing with the first. `examples/timbre` keeps one, because **there** it is what separates "nobody spoke" from "the wrong person did".

## 10 What has actually been measured

Two kinds of check run against these two networks, and conflating them is the error this section is arranged to prevent.

## 10.1 Coverage proves a tree, never an arithmetic

MDX23C loads at 319/0/0 and CAM++ at 815/0/122. Both numbers say the module tree matches the checkpoint, and both would be **unchanged** by the U-net running with frequency as the image height, by a batch norm where an instance norm belongs, or by a filterbank missing its mean subtraction. This repository has already shipped a port that loaded at 100% and produced garbage. So each network needs a second check that exercises arithmetic, and each has one.

A note on reading the third column: two conventions are in use across this workspace and they are opposites. `burn-whisper`'s load prints the **genuinely** unused count with its known allowance already subtracted; `burn-campplus`'s prints the **raw** count and accounts for it by name afterwards. Comparing MDX's 0 against CAM++'s 122 and concluding one port is cleaner is the wrong conclusion, not a small one.

## 10.2 Synthetic: the check that proves the port

`burn-mdx`'s `examples/separate` mixes a known voice with a known bed, so every reading has a reference and SI-SDR against a **source** is available. Measured on LibTorch/CUDA over six clips of this repository's own corpus against a synthesised organ chord at a 0 dB mix:

| reading | value |
|---|---|
| **partition** — `est_vocals + est_instrumental` against the mixture | **41.5 dB** SI-SDR |
| solo voice in — vocals / instrumental rms | 0.0516 / 0.0299 (**4.7 dB**) |
| solo bed in — vocals / instrumental rms | 0.0226 / 0.0526 (**7.3 dB**) |
| SI-SDR of `est_vocals` against voice / against bed | −0.25 / −0.54 dB |
| envelope corr, `est_vocals` against voice / bed | 0.678 / 0.645 |
| envelope corr, `est_instrumental` against voice / bed | 0.508 / 0.583 |

Figure 7: The synthetic mode. **Read the first row first.**

The two stems reconstruct their input to 41.5 dB, and the solo probes split **in opposite directions** depending on which source went in. Together those say the forward pass is coherent and content-dependent: a transposed U-net, a batch norm where an instance norm belongs, a mis-scaled block or a scrambled stem axis destroys one or both, because nothing downstream re-imposes either property.

**What it does not establish is separation quality**, and the numbers are honest about that: 4.7 dB of rejection is far below the 20 dB-plus a vocal separator manages on the material it was trained for, and every mixture row sits within a decibel of doing nothing. The reason is the input rather than the port — MDX23C was trained on **sung** vocals inside real productions, and this feeds it dry, close-mic speech over a synthesised chord, which is out of distribution on both sides.

Note also what the baseline is. `burn-gptsovits`'s `reconstruct` correlates its output against its **input**, which is right for a round trip and wrong here: a network that returned its input unchanged would score beautifully. For separation the meaningful do-nothing is the **unprocessed mixture**, so it is a row of every table and the number that means anything is the delta from it.

## 10.3 Real: the check that measures usefulness

`--mixture <file>` is the reference-free mode. No stems exist — nobody holds the sources of somebody else's stream — so no SI-SDR against a source is reported and every reading is a contrast the mixture is measured under the same way. One 19.8-minute stereo stream at 44.1 kHz, a streamer talking over somebody else's music, LibTorch/CUDA at 50% overlap-add. `contrast` is the loudest

against the quietest tenth of 250 ms frames, chosen by the **mixture** so all three columns are read over the same instants; `bed out` is how far `est_vocals` sits under the mixture in those quiet frames, which is the music that came out of it.

| **excerpt** | side below mid | partition | vocals rms | instr rms | contrast mix / voc / instr | bed out |
|---|---|---|---|---|---|---|
| 900–960 s | 18.6 dB | 34.9 dB | −2.5 dB | −5.1 dB | 19.9 / **24.4** / 10.5 dB | **−6.5 dB** |
| 900–930 s | 17.1 dB | 36.3 dB | −3.0 dB | −5.2 dB | 22.1 / **25.7** / 11.6 dB | −6.0 dB |
| 180–210 s | 14.3 dB | 34.2 dB | −2.6 dB | −3.5 dB | 12.9 / **16.8** / 7.7 dB | −5.5 dB |
| 480–510 s | 16.2 dB | 34.6 dB | −0.7 dB | −7.3 dB | 22.5 / 23.5 / 16.7 dB | −2.0 dB |
| 900–960 s, mono fold | — | 37.5 dB | −2.6 dB | −5.4 dB | 19.9 / 22.9 / 10.7 dB | −5.1 dB |

Figure 8: The real-recording mode. **These numbers and the synthetic ones answer different questions and must not be merged.**

**It separates, and the amount is modest.** The three contrast columns move together in the way a working separation has to: the vocals stem's contrast comes out **above** the mixture's and the instrumental stem's **below** it, which is one stem following the intermittent speech and the other following the continuous bed. Neither stem is near-silent. But 5–6.5 dB of bed removal is a long way from the 15–20 dB this model reaches on a song, so the music is attenuated rather than gone — and driven through `preprocess separate` rather than the example, one of those excerpts reads 4.1 dB.

Four things about that table not to re-derive:

- **The 480 s row is the control, not an outlier.** That region's bed stops between phrases instead of running under them, so there is little bed in the quiet frames to remove — and the instrumental stem's contrast rises to 16.7 dB, tracking a bed that is itself intermittent. Less removal where there is less to remove is the model behaving, and it is why one excerpt is not a measurement.
- **Read the gaps at 250 ms, not at one second.** A between-sentence gap is a few hundred milliseconds, so at a one-second window no "quiet" frame is speech-free and the verdict **inverts**: the same stems on the same 60 s give 2.1 dB of removal and a vocals contrast **below** the mixture's, which reads as a model that separated nothing.
- **Near-mono is not the explanation.** The side channel sits 14–19 dB under the mid on every excerpt, which removes most of the spatial cue a stereo-native separator would use. Folding to true mono and re-running takes the removal from 6.5 dB to 5.1 dB — real, and far too small to be the gap. What is left is the material: a speaking voice is not a sung one. Which also means a **mono corpus loses almost nothing** by going through this stage.
- **The partition is 34–37 dB here against 41.5 dB on one chunk.** Overlap-add seams are part of that — the synthetic mode runs a single chunk with no seam at all — but demonstrably not all of it: the mono fold has the same file, the same hop and therefore the same seams, and reads 37.5 dB against the stereo run's 34.9. So the partition reading is content-sensitive, which is worth knowing before treating a change in it as a regression.

**Every figure above reads the mono downmix, and the stems are stereo.** Folding both the mixture and the stem to mid puts the bed 4.1 dB down in that excerpt's speech gaps; reading the same two files as written gives 1.0 dB, because what the stem keeps of the bed is largely out of phase between the channels and cancels in the fold, where the mixture's own level barely moves. Neither reading is wrong and they are not interchangeable — the mono one is what was recorded, and it is also the one that matters here, since everything downstream decodes mono.

### 10.4 The finding that is not in decibels

Transcribing the same 60 s of mixture and of vocals stem with the same flags:

- **The words are the same** — same content, same order, differing in one character across the minute.
- **The segmentation is not, and that is the finding.** The mixture yields **5** segments, one of them a single merged run of most of a phrase; the stem yields **14**, one per utterance. The slicer cuts on silence, and a continuous bed means the recording **has** no silence — so a corpus behind music cannot be sliced into sentences at all until the bed comes off.
- Sliced with `clip`'s defaults, the same 60 s gives **5** clips holding 58 of its 60 seconds before separation, and **12** holding 39 after. The mixture has no sentence boundaries to find, so it "keeps" almost everything as one lump.
- **The stem also invents.** One of the 14 was a clear hallucination — English in a Chinese-pinned run — and one was a 0.37 s fragment, both in near-silent frames the mixture's longer segments had swallowed. Emptier gaps give a recogniser more room to invent, so anything consuming these stems wants a duration or confidence floor.

**Pin `--language`.** Left to detect, the mixture and the stem disagree about which language they are, and the comparison stops meaning anything.

### 10.5 One place the record contradicts itself

The longest merged segment in that mixture transcription is recorded as **18.8 s** in `burn-mdx`'s `examples/separate` and as **16.9 s** in `preprocess-core`'s `separate` module doc. The segment **counts** (5 against 14) agree everywhere, as do the words. No re-run has resolved which duration is right, so it is named here rather than averaged, picked or quietly dropped — the same treatment the workspace gives its `815/0/0` against `815/0/122` disagreement.

## 11 Decisions, and the traps behind them

### 11.1 `preprocess-core` depends on `cli-kit`, and that is allowed

A `*-core` crate depending on a CLI crate reads as a layering slip and is not one. `backend::load` boxes a `dyn Separator` and `embed::load` a `dyn SpeakerEmbedder`, so both have to **name** a backend — and the workspace rule is that there is exactly one `--backend` enum, in `cli-kit`. Declaring a second one here to avoid the dependency is the thing explicitly forbidden; `cli-kit` is a `*-kit` crate, which anything may depend on. This is `seedvc-core`'s precedent generalising rather than an exception spreading, and the test is whether the crate erases a Burn backend behind a trait object, not which crate it is. Fetching weights stays with the caller, so nothing here touches `hub-kit`.

The two erasures sit in different modules — the separator's in `backend`, the embedding's in `embed` — and that is chronological rather than principled. The first was written when it was the only model; the second sits beside the trait it erases, because a second `resolve` and a second `load` in one module would each have to be spelled differently from the first, and each refusal names its own model, so there is nothing to share but the shape.

**Neither stage's arithmetic pays for the backend.** The two traits, the overlap-add, the window arithmetic and both file drivers compile and are tested with **no** backend enabled; only the loader arms that name a Burn backend sit behind `cuda` / `tch` / `wgpu`. That split is what keeps the seam and the segment arithmetic under `cargo test` on a machine with no checkpoint and no GPU, and what lets a build with no backend at all still **refuse** a request with a reason rather than failing to have the function.

## 11.2 Both model stages refuse before they fetch

`separate` validates its backend and its download policy **before** the 448 MB the model weighs, and `diarize` does the same plus its window arithmetic — a `--hop` larger than `--window` skips audio nothing ever scores, and it does so silently. Neither check is there for tidiness: a backend this build cannot run and a download policy that abandons every transfer are equally cheap to notice now and equally expensive to notice after the fetch.

## 11.3 This crate was the fourth to arrive with a known bug

`burn_store`'s applier derives `missing` as **visited and not applied and not skipped and not errored**, so a path that failed to apply is dropped from `applied` and from `missing` alike — and a checkpoint carrying the right tensor **names** at the wrong **shapes** reads as a clean load while every affected parameter sits at its initialised value. `burn_kit::check_coverage` asks in the right order: `errors` first, then `missing`, then `applied.is_empty()`.

That fix enumerated eight call sites. `preprocess-core` was written in a parallel worktree while it was going through, so it was never one of the eight, and it arrived with a hand-written `missing.is_empty()` in each of its two loaders. **A fix by enumeration cannot reach code that does not exist yet.** The question to ask of a new loader is therefore not "was it in that list" but **does it call `check_coverage`** — `grep check_coverage` over a crate that loads weights is the whole audit.

## 11.4 The shared target directory

`target/debug/examples/<name>` is not hashed per worktree, so two checkouts building an example of the same name overwrite each other's binary and `cargo run --example` silently executes whichever landed last. That is not hypothetical here: one worker got a complete, plausible and **wrong** coverage report exactly this way while Seed-VC was being ported. Every number in this document that came from an example was produced with an isolated `CARGO_TARGET_DIR`, and anything re-measuring them needs the same.

## 11.5 The stage boundary is a file, deliberately

Nothing here streams between stages. Each reads audio files and writes audio files, which costs a decode and an encode per stage and buys the property the whole phase rests on: any stage's output is a legitimate input to any other, in any order, and a user can look at what came out before spending an hour on the next one. Three of the eight stages are knobs a user is expected to get wrong on the first attempt; that is not a pipeline to hide inside a single process.

## 12 The recipes

```
# 0. What is this corpus? No model, no download, nothing written.
preprocess analyze raw/                    # and read `continuous` first

# 1. If it says "no dead air at any floor": the bed has to come off.
preprocess separate raw/ -o vocals/        # MDX23C, 44.1 kHz stereo out

# 2. If more than one voice survives that (a singer on the backing track):
preprocess diarize vocals/ -o mine/ --reference streamer.wav

# 3. Level and hiss, in whichever order suits the material.
preprocess denoise   mine/ -o clean/
preprocess normalize clean/ -o level/ --lufs -23

# 4. Cut into sentences. This is what a trainer eats.
preprocess clip level/ -o dataset/ --silence-db -45

# 5. Check what that produced, at the flags that produced it.
preprocess analyze dataset/ --silence-db -45 --per-file
```

Figure 9: What to run, in the order the decisions come.

Two orderings are worth stating because they are not obvious:

- **`analyze` twice.** Once before anything, to choose the floor and to find the recordings that cannot be sliced at all; once at the end, against the very flags `clip` was run with, to see the clip-length histogram and the silence ratio the trainer will actually meet.
- **`separate` before everything.** It is the only stage whose absence makes a later one impossible rather than merely worse — and it is also the only stage whose output rate and channel count are not a choice, so a `resample` after it is the normalising step rather than one before.

`trim` and `resample` do not appear above because they are the stages a corpus needs when it is **nearly** right: `trim` for recordings that are already one utterance each and only have dead ends, `resample` for a corpus arriving at three different rates in four container formats.

---

**Sources.** Module documentation of `preprocess-core` (`lib`, `analyze`, `clip`, `denoise`, `diarize`, `embed`, `normalize`, `resample`, `separate`, `trim`), `preprocess-cli` (`lib`, `args`), `burn-mdx` (`lib`, `net`, `stft`), `burn-campplus` (`lib`, `fbank`) and `audio-kit`'s `slice`. Measurements from `burn-mdx`'s `examples/separate` (both modes) and `preprocess-core`'s `examples/timbre`, which are where those numbers stay current. Every figure here is copied from one of those and nothing is estimated. The two places the record disagrees with itself are named where they arise rather than smoothed over: a merged segment's duration (Section 10.5) and `analyze`'s second reason for having no loudness column (Section 3.3).